

第七章 图

知识点一：图的定义和术语

知识点二：图的存储结构

知识点三：图的遍历

知识点四：连通网的最小生成树

知识点五：拓扑排序

知识点六：关键路径

知识点七：最短路径

知识点四 连通网的最小生成树

内容简介

- 1、无向图的连通分量和生成树;
- 2、最小生成树定义及性质;
- 3、最小生成树相关应用;
- 4、最小生成树典型算法;
- 5、总结.

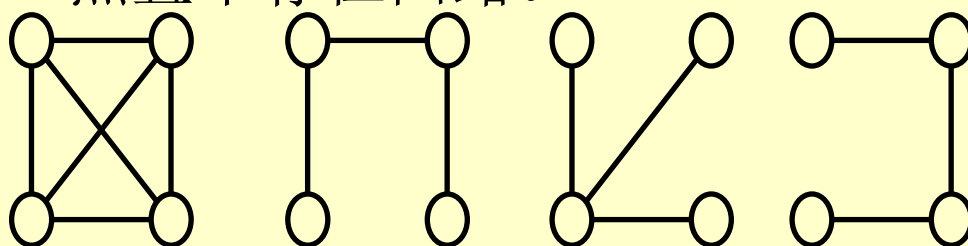
1、无向图的连通分量和生成树

- 判定图的连通性方法：利用遍历图算法可以求解图的连通性问题
 - 对于无向连通图：从图中任一顶点出发，进行深度优先搜索或广度优先搜索，便可访问到图中所有顶点。
 - 对于非连通图：从多个顶点出发进行搜索，而每一次从一个新的起始点出发进行搜索过程中得到的顶点访问序列为其各个连通分量中的顶点集。
 - 判定一个无向图是否为连通图，利用计数变量统计搜索算法覆盖所有顶点所需调用次数，该次数即为连通分量数。

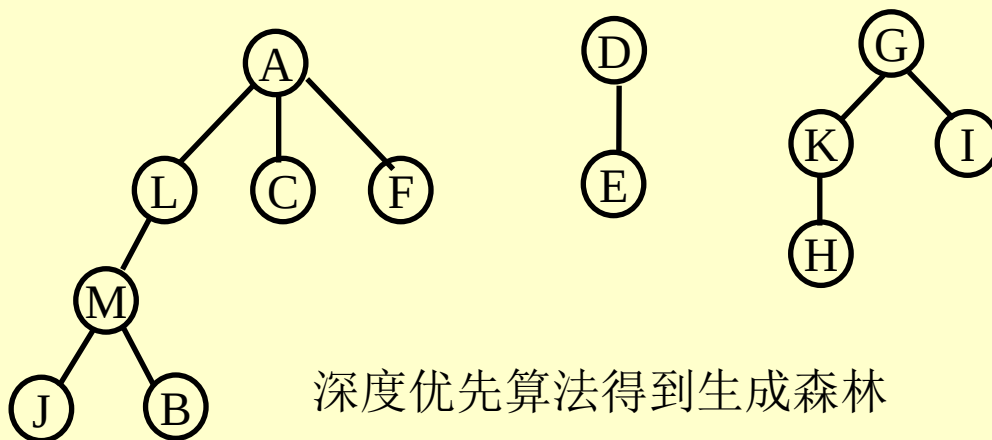
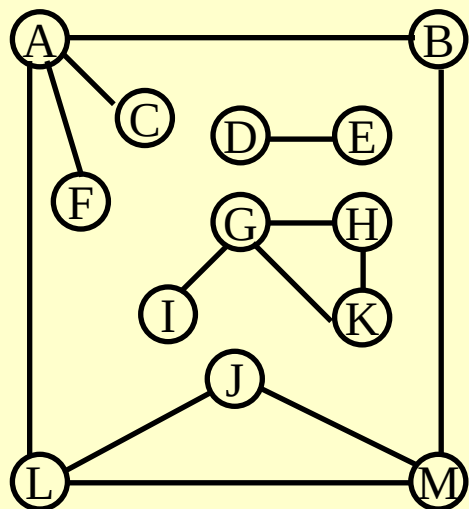
无向图的连通分量和生成树

➤生成树与生成森林：无向连通图可采用遍历算法得到生成树，非连通图可得到生成森林。

生成树：图G的一棵生成树是一个极小连通子图，它连接图中所有顶点且不存在回路。



生成森林：非连通图每个连通分量的生成树一起组成非连通图的生成森林。

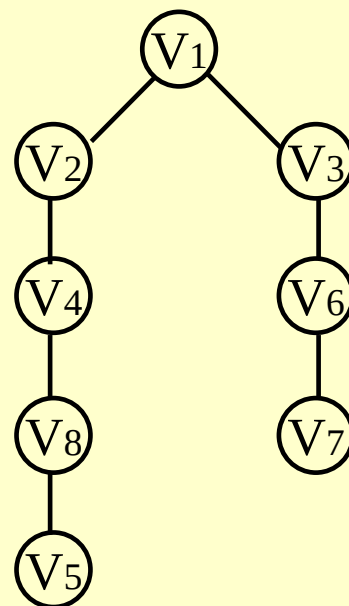
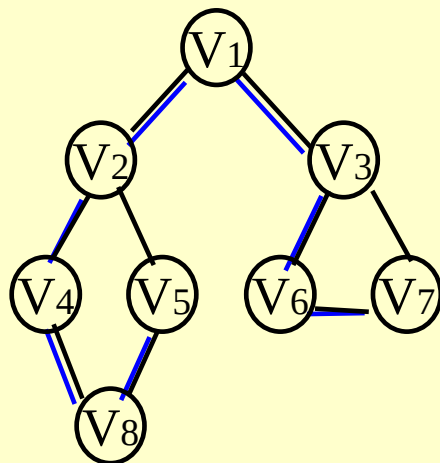
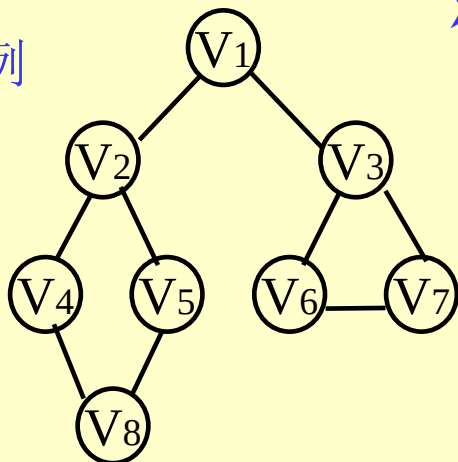


深度优先算法得到生成森林

无向图的连通分量和生成树

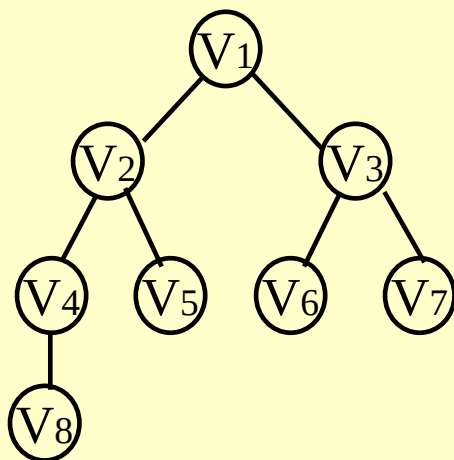
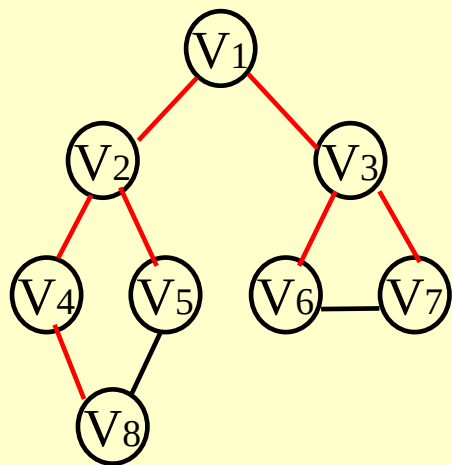
例

➤ 深度遍历: $V1 \Rightarrow V2 \Rightarrow V4 \Rightarrow V8 \Rightarrow V5 \Rightarrow V3 \Rightarrow V6 \Rightarrow V7$



深度优先生成树

➤ 广度遍历: $V1 \Rightarrow V2 \Rightarrow V3 \Rightarrow V4 \Rightarrow V5 \Rightarrow V6 \Rightarrow V7 \Rightarrow V8$



广度优先生成树

生成树的意义: 若从图的某顶点出发, 可以系统地访问到图中所有顶点, 现实应用中无明确意义。

2、最小生成树定义及性质

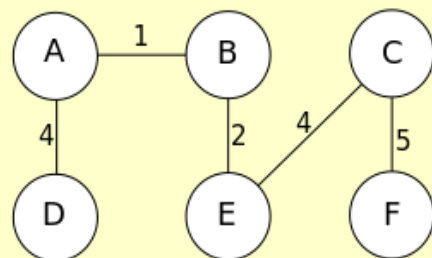
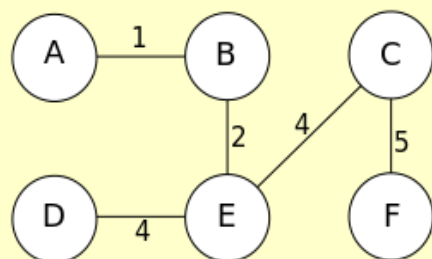
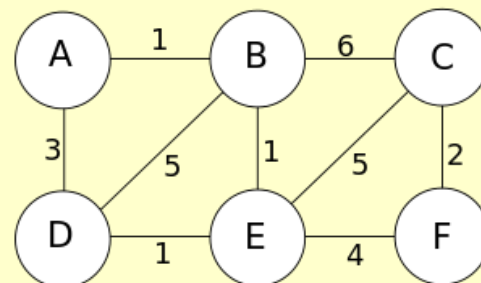
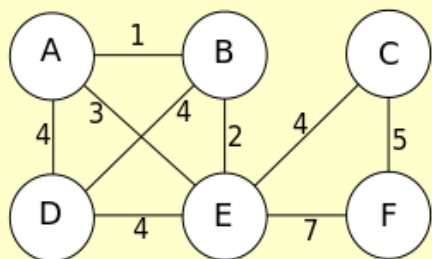
最小生成树:带权连通图(连通网)中代价(边权)和最小的生成树 (Minimum Spanning Tree, MST)

- 树: 无回路;
- 最小: 边上的代价和最小;
- Spanning: 覆盖所有节点, 所需边数为 $|V| - 1$;
- 连通: 存在性条件;
- 结构最小性: 加入非树上的边会得到一个回路.

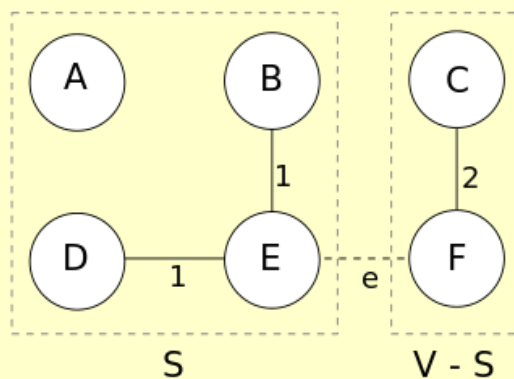
最小生成树定义及性质

性质1: 唯一性(权值无重复时)
(unique)

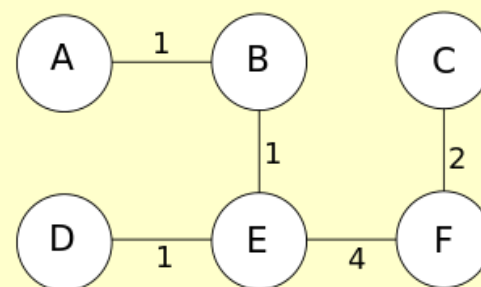
http://en.wikipedia.org/wiki/Minimum_spanning_tree



The cut:



MST T:

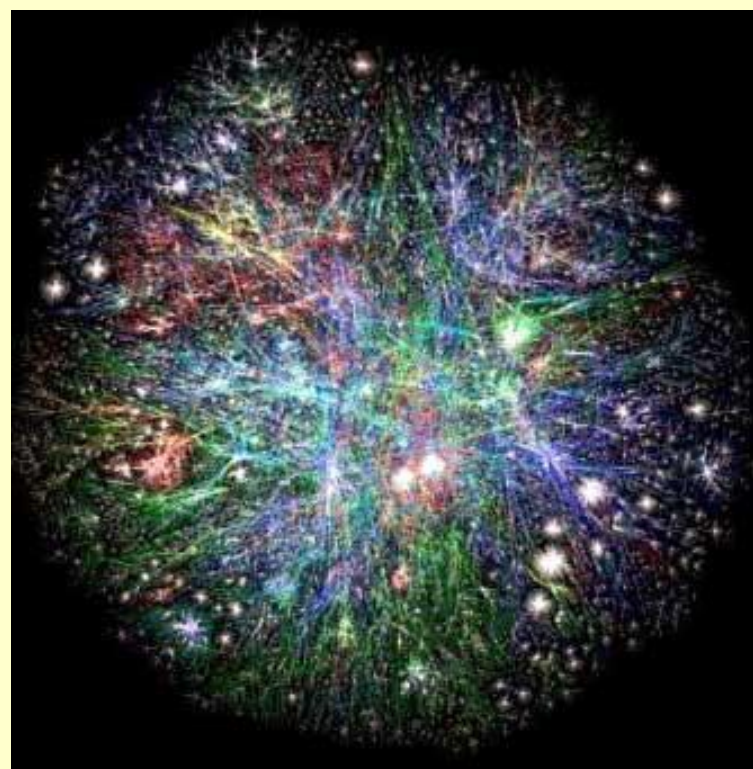


性质2: 切割属性(cut property)

给定连通网 $N=(V, \{E\})$, S 为顶点集 V 的非空子集, 若 e 为 S 与 $V-S$ 切割边中是代价最小者, 则 $S \cup \{e\}$ 为MST中的一部分。(反证法证明)

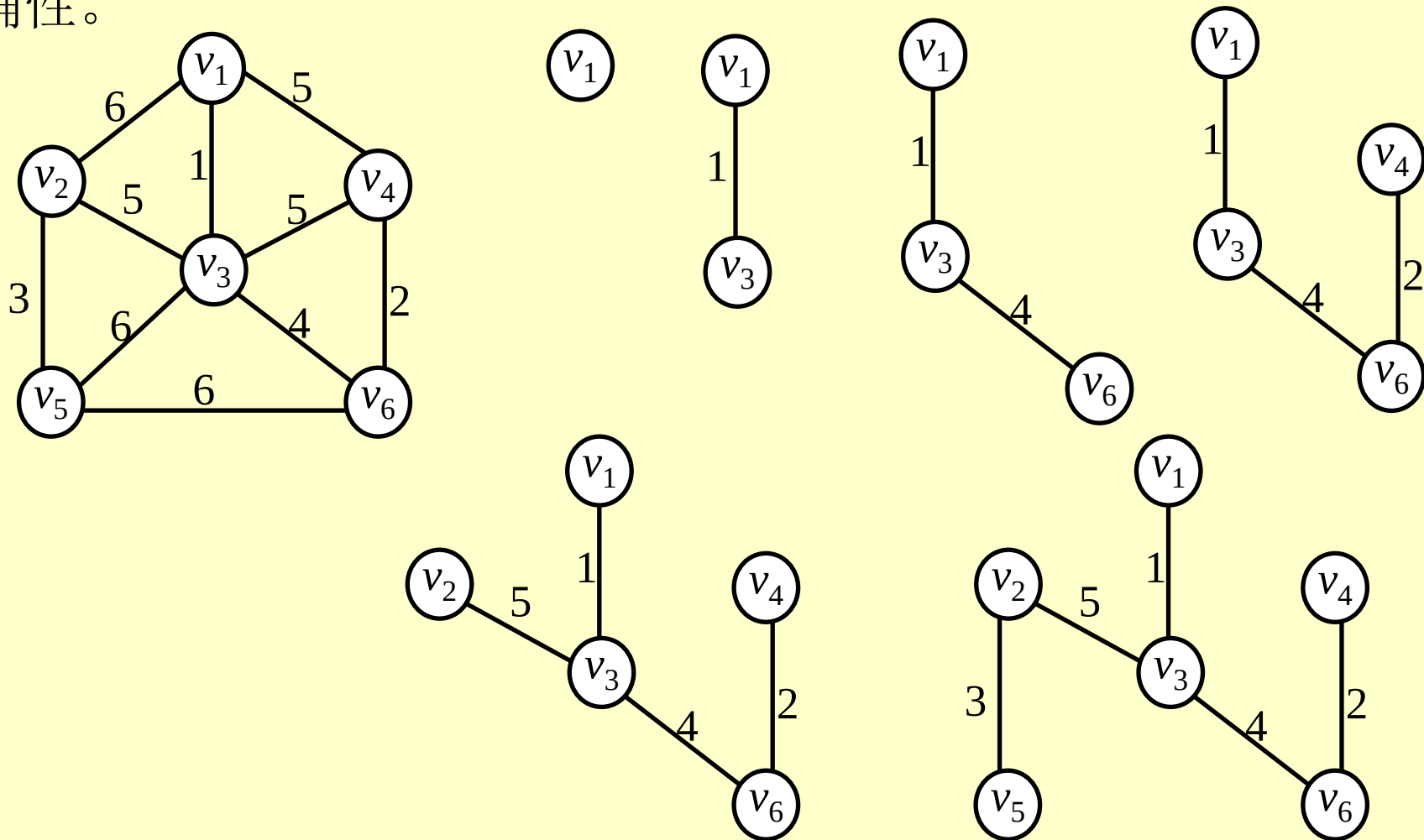
3、最小生成树相关应用

- 直接应用：交通网络、计算机网络、电信网、供水网络及电子网格等的网络规则设计，使代价和最小；
- 间接应用：相关NP难问题的近似求解方法，如社会网络中的沟通代价计算；另外，分类系统、聚类分析、手写识别、电路设计、生物信息学等领域均用到了MST概念。



4、最小生成树典型算法

➤ 普里姆(Prim)算法思想：贪心策略、选择顶点、树增长方式、每次选择与当前结果相连且边权最小的顶点，切割性质保证其正确性。



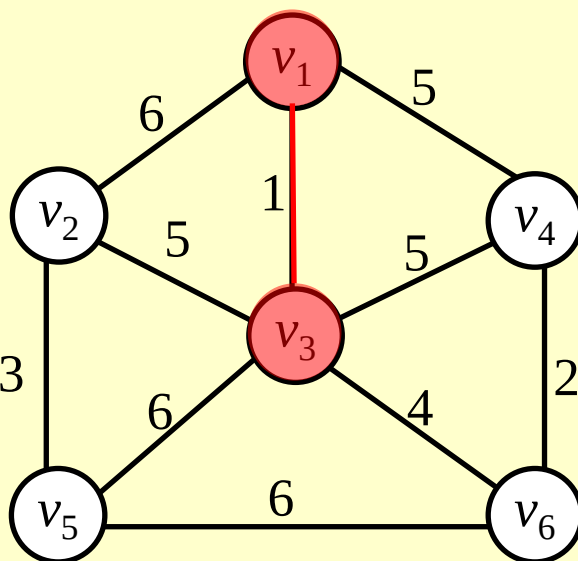
最小生成树典型算法

➤ 普里姆算法伪码

- 输入连通网 $N=(V, \{E\})$;
- 初始化 $U=\{v_1\}$ (v_1 为 V 中任意结点), $T=\{\}$;
- while ($U \neq V$)
 - 在集合 E 中选取权值最小的边 (u, v) , 其中 $u \in U, v \in V-U$;
- 将 v 加入 U , 边 (u, v) 加入 T ;
- 输出 U 、 T .

$U=\{v_1, v_3\}$

$V-U=\{v_2, v_4, v_5, v_6\}$



问题: $V-U$ 中的顶点与 U 中多个顶点相连, 是否需要保存所有这些边权?

➤ 普里姆算法实现

(1) 网的存储结构选择：邻接矩阵

```
#define MAXVEX 12
#define INFINITY 65535
typedef char VertexType; //顶点数据类型
typedef int EdgeType; //边表的权值类型
typedef struct graph{
    VertexType data[MAXVEX]; //图的顶点
    EdgeType Edge[MAXVEX][MAXVEX]; //图的边表
    int NumVertex,NumEdge; //图的顶点数与边数
}Graph;
```

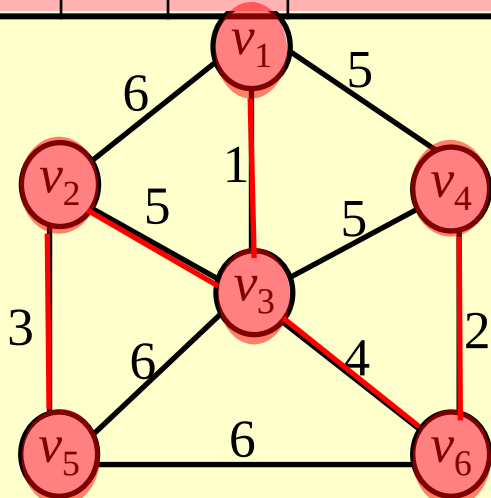
(2) V- U中顶点到U最小边权保存：设置一个一维数组closedge[MAXVEX]，保存V- U中各顶点到U中顶点具有最小权值的边。也可用两个单独的数组。

```
struct {
    int adjVex ;    /* 最小权值边所连接的U中顶点*/
    int lowcost ;   /*存放最小的权值*/
} closedge[MAXVEX];
```

最小生成树典型算法

Vex closededge	v ₂	v ₃	v ₄	v ₅	v ₆	U	V-U	k
adjVex lowcost	v ₁ 6	v ₁ 1	v ₁ 5			{v ₁ }	{v ₂ , v ₃ , v ₄ , v ₅ , v ₆ }	3
adjVex lowcost	v ₃ 5		v ₁ 5	v ₃ 6	v ₃ 4	{v ₁ , v ₃ }	{v ₂ , v ₄ , v ₅ , v ₆ }	6
adjVex lowcost	v ₃ 5		v ₆ 2	v ₃ 6		{v ₁ , v ₃ , v ₆ }	{v ₂ , v ₄ , v ₅ }	4
adjVex lowcost	v ₃ 5			v ₃ 6		{v ₁ , v ₃ , v ₆ , v ₄ }	{v ₂ , v ₅ }	2
adjVex lowcost				v ₂ 3		{v ₁ , v ₃ , v ₆ , v ₄ , v ₂ }	{v ₅ }	5

➤ 普里姆算法实例



问题： 当新顶点加入U后，如何更新V- U中的顶点与U中相连边权最小值？

只与新加入顶点相关

```
void MiniSpanTree_Prim(Graph *G)
```

```
{   int i,j,k,min;
```

```
    int adjVex[MAXVEX]; //存放顶点的下标值
```

```
    EdgeType lowcost[MAXVEX]; //存放最小的权值
```

```
    adjVex[0] = 0; //选取下标为0的一个顶点
```

```
    lowcost[0] = 0; //表示此顶点已经被处理过
```

```
    for(i = 1; i < G->NumVertex; ++i)
```

```
    {   _____; //将与下标为零的顶点邻接的边权值存放在lowcost中
```

```
        adjVex[i] = 0; //全部初始化为下标为零的顶点
```

```
    }
```

```
    for(i = 1; i < G->NumVertex; ++i) //选择其余G.vexnum-1个顶点
```

```
    {   min = INFINITY; //初始化最小值为无穷大
```

```
        k = 0; //用于记录权值最小的顶点的下标值
```

```
        for(j = 1; j < G->NumVertex; ++j) //求出下一个结点，用k记录下标
```

```
        {   if(_____ && _____) //如果顶点没有处理过而且权值小于最小值
```

```
            {   min = lowcost[j]; //把权值赋给min
```

```
                k = j; //记录当前最小权值的顶点的下标
```

```
            }
```

```
        }
```

```
        lowcost[k] = 0; //标记下标k的顶点为已处理
```

```
        printf("(%d %d)\n", adjVex[k], k);
```

```
        for(j = 1; j < G->NumVertex; ++j) //更新V- U中顶点连接U中顶点最小边权边, 只与新加入顶点k相关
```

```
        {   if(_____ && _____) //如果顶点没有处理过且权值小于此前未被加入时的顶点权值
```

```
            {
```

```
                _____; //将较小的权值存入lowcost的相应位置，与k相关。
```

```
                _____; //在adjVex数组中记录下顶点的下标
```

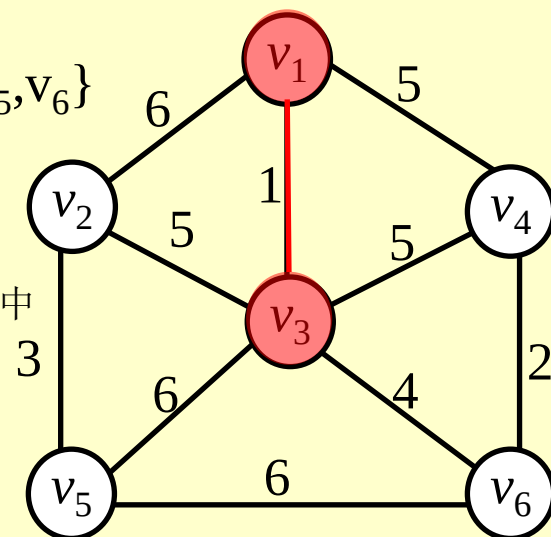
```
            }
```

```
        }
```

```
    }
```

```
}
```

$$U = \{v_1, v_3\}$$

$$V - U = \{v_2, v_4, v_5, v_6\}$$


➤ 算法分析

(1) 时间复杂度：设带权连通图有 n 个顶点，则算法的主要执行是两重循环

- 求权值最小的边，代价 $n-1$ ；

- 修改最小权值数组，代价为 n 。

整个算法的时间复杂度是 $O(n^2)$ ，与边的数目无关，适用于稠密网。

(2) 空间代价：除网结构存储外，辅助空间为 $2n$ 。

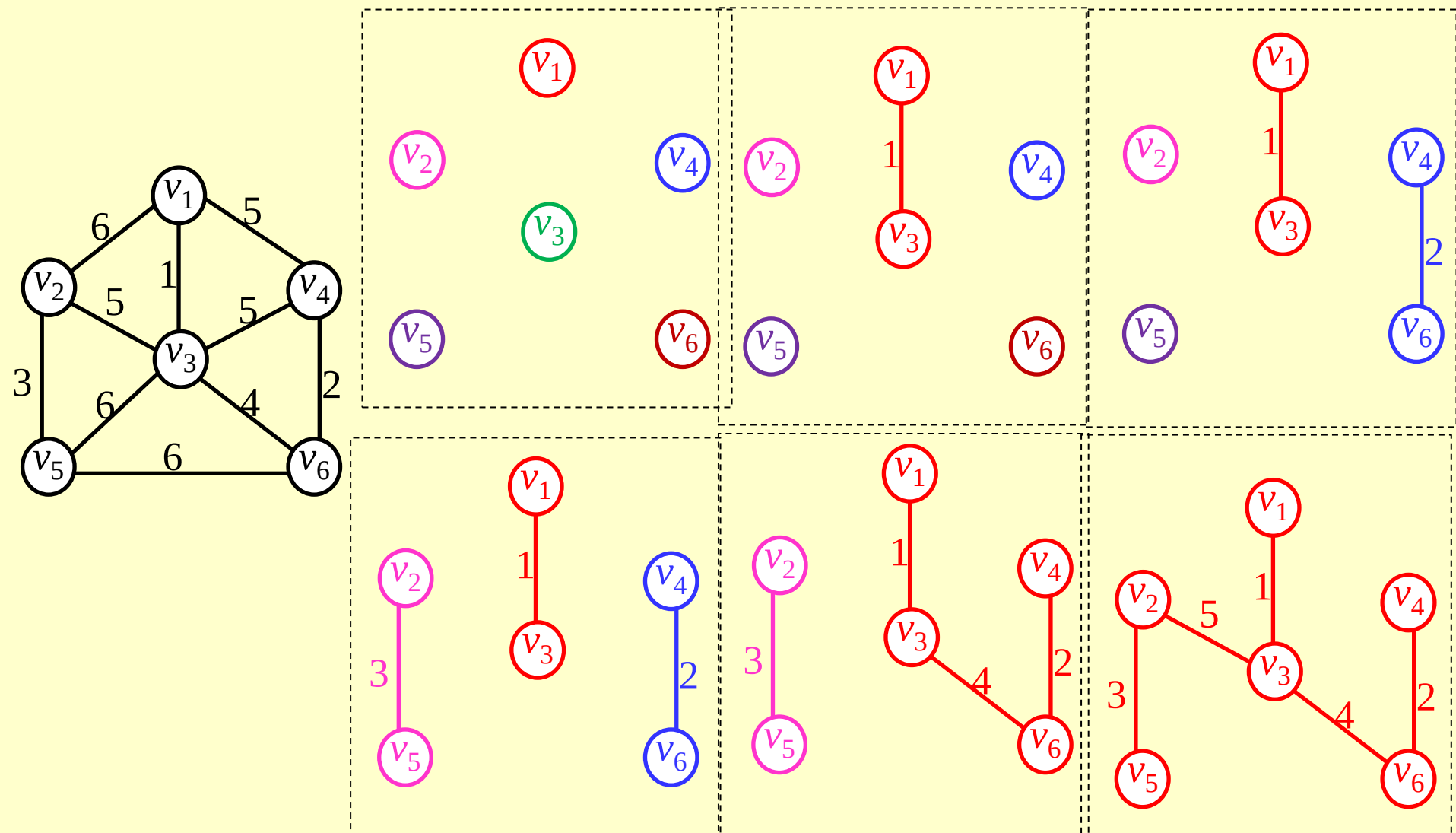
Experiment 7-2 MST Prim

Problems:
MST Prim;

最小生成树典型算法

3 典型算法之克鲁斯卡尔 (Kruskal) 算法

- 算法思想：贪心策略、选择边、连通分量合并方式、每次选择权值最小的边去合并不同的连通分量，切割性质保证正确性。



➤ 克鲁斯卡尔算法伪码

- (1) 设连通网 $N=(V, \{E\})$ ，令最小生成树为 T ；
- (2) 初始状态为只有 n 个顶点而无边的非连通图 $T=(V, \{\Phi\})$ ，每个顶点自成一个连通分量；
- (3) 在 E 中选取代价最小的边，若该边依附的顶点落在 T 中不同的连通分量上，则将此边加入到 T 中；否则，舍去此边，选取下一条代价最小的边；
- (4) 重复(3)，直到 T 中所有顶点都在同一连通分量上。

➤ 算法实现 用顶点数组和边数组存放顶点和边信息

<pre>typedef struct{ int vexNum; //顶点信息 int compNum; //连通分量编号 }Vex[n]; 或一单独数组int *index; //index数组, 其元素为连通分量的编号</pre>	<pre>typedef struct { int vexHead, vexTail ;//边依附的两顶点 int weight; //边的权值 int flag; //标志是否连接两个不同的连通分量 }Edge;</pre>
---	---

回路判断： 定义一个顶点数组，顶点中compNum存放每个顶点所在的连通分量的编号。

(1) 初值： $Vex[i].compNum = i$ ，每个顶点各自组成一个连通分量，连通分量的编号设为顶点编号

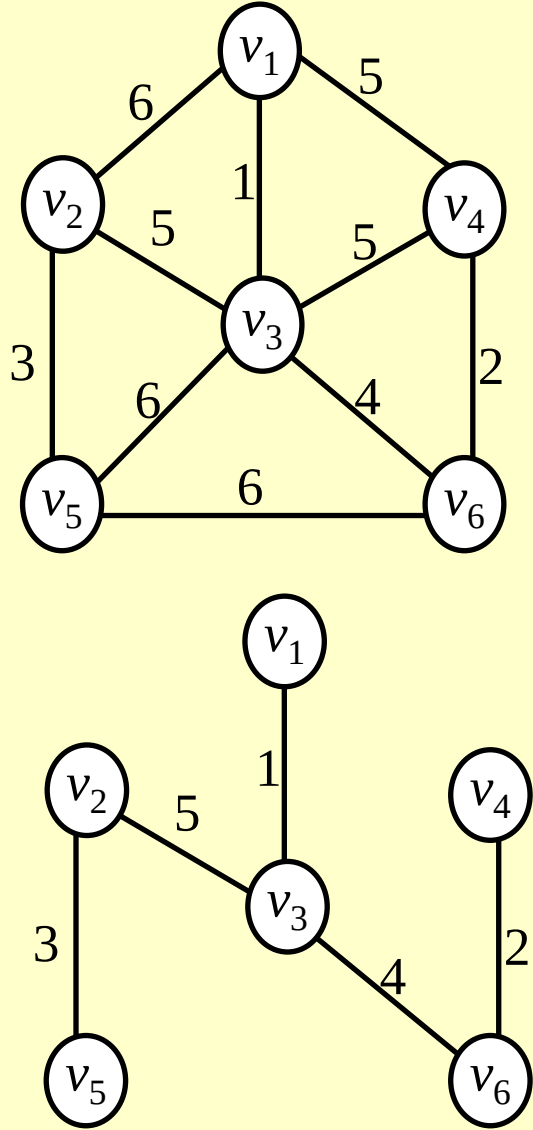
(2) 当往T中增加一条边(v_i, v_j) 时，先检查 $Vex[i]$ 与 $Vex[j]$ 的compNum值：

若 $Vex[i].compNum == Vex[j].compNum$ 则 v_i 和 v_j 在同一个连通分量中，加入此边会形成回路；

若 $Vex[i].compNum \neq Vex[j].compNum$ 则加入此边不会形成回路，将此边加入到生成树；

(3) 加入一条新边后，将两个不同的连通分量合并，用一个编号表示两个连通分量。

➤ 克鲁斯卡尔算法实例



VexNum compNum

1	1	1	2
2	2	2	
3	3	3	1 2
4	4	4	1 2
5	5	5	2
6	6	6	4 1 2

vexHeadvexTailweight flag

0	1	2	6	0
1	1	3	1	0
2	1	4	5	0
3	2	3	5	0
4	2	5	3	0
5	3	4	5	0
6	3	5	6	0
7	3	6	4	0
8	4	6	2	0
9	5	6	6	0

➤ 克鲁斯卡尔算法

```
void Mst_Kruskal ( Graph G ) {  
1   T = { } ;  
2   while ( T 中少于 $|V| - 1$  条边 && E 非空 ) {  
3       从E中选择代价最小的边( $v_i, v_j$ ) ;  
4       从E中删除 ( $v_i, v_j$ ) ;  
5       if (( $v_i, v_j$ ) 在T中不会形成回路 )  
6           将( $v_i, v_j$ )加到T中 ;  
7       else  
8           放弃( $v_i, v_j$ ) ;  
9   }  
10  if ( T 包含少于 $|V| - 1$ 条边 )  
11      Error ( “无最小生成树” ) ;  
}
```

➤ 算法分析:

边表按权值采用堆排序或快速排序, 时间复杂度是 $O(|E|\log|E|)$, 与顶点数无关, 适用于稀疏网。

```
#define MaxSize 20
typedef char VertexType;

typedef struct Graph {    //定义图的邻接矩阵表示法结构
    VertexType ver[MaxSize+1];
    int edg[MaxSize][MaxSize];
}Graph;

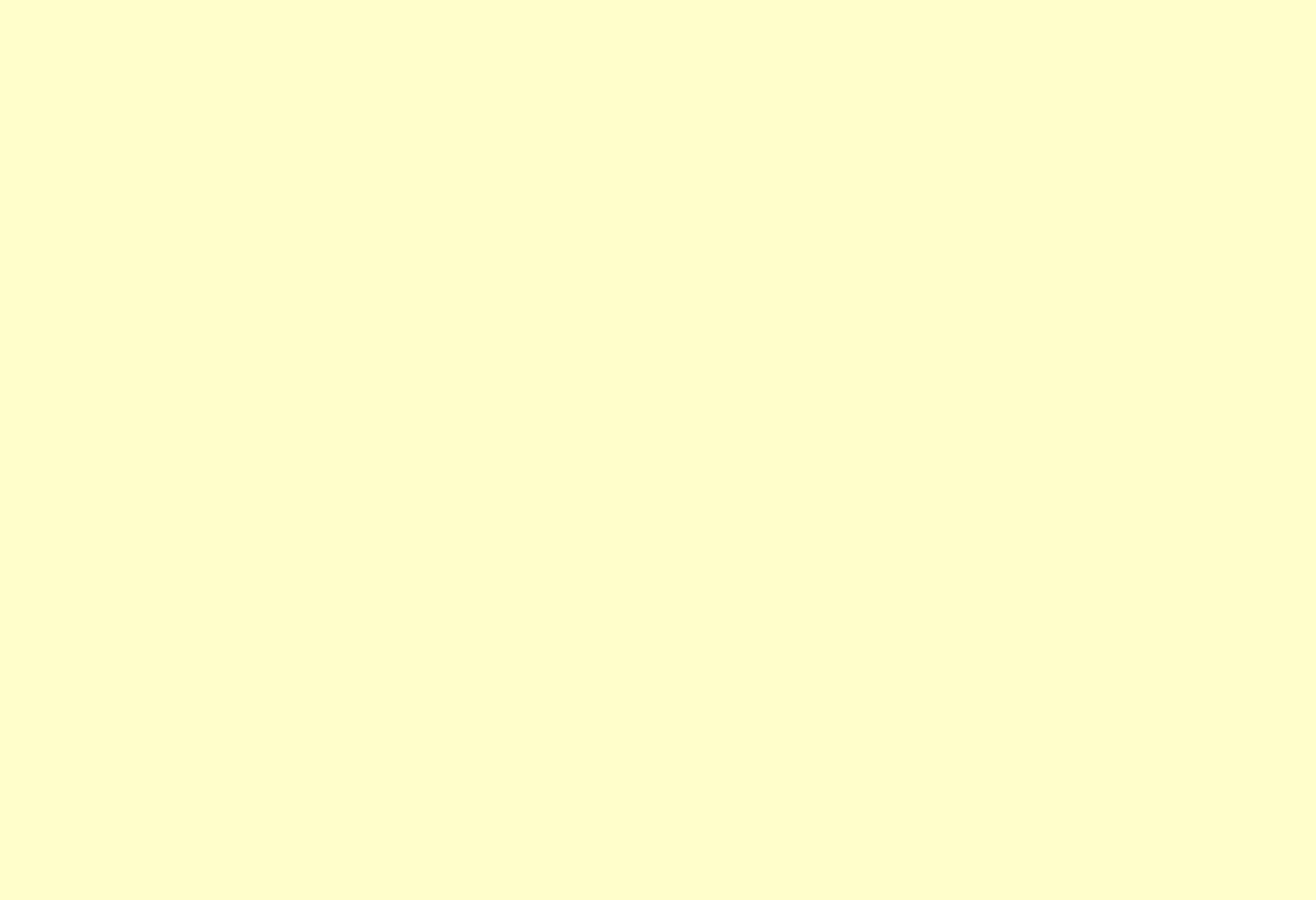
typedef struct gEdge {    //定义一个边集结构，用来存储图的所有边信息
    int begin;
    int end;
    int weight;
}gEdge;

int CreateGraph( Graph *g )    //邻接矩阵法图的生成函数

gEdge *CreateEdges( Graph g ) //生成图的排序过的边集数组

void Kruskal( Graph g ) //Kruskal算法生成MST
int *index; //index数组，其元素为连通分量的编号，index[i] == index[j] 表示编号为i和j的顶点在同一个连通分量中，反之则不在同一个连通分量
```

代码展示

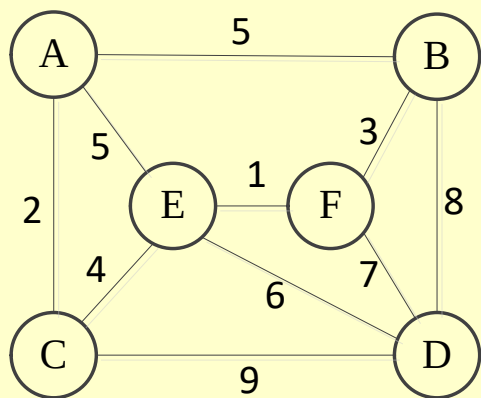


Experiment 7-MST Kruskal

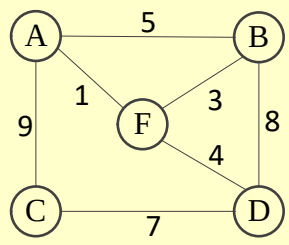
Problems:

Mst Kruskal

- 1 给定连通网结构如图所示，利用普里姆(Prim)算法，求出最小生成树，起始结点为A，给出每一步结果示图。
- 2 克鲁斯卡尔(Kruskal)算法，求出最小生成树，给出每一步结果示图。



设无向图G（如下图所示），则其最小生成树上所有边的权值之和为_____。



5、总结

算法	生成方式	存储结构	辅助空间	时间复杂度	适用情况
Prim	选顶点	邻接矩阵	N	$O(N^2)$	稠密网
Kruskal	选边	邻接表、边集数组	N	$O(E \log E)$	稀疏网

相关问题

- k-minimum spanning tree (k-MST) : spans some subset of k ;
- Steiner tree: spans the given subset;
- Euclidean minimum spanning tree: vertices are points in the plane;
- Capacitated minimum spanning tree: contains no more than a c nodes;
- Degree constrained minimum spanning tree: each vertex is connected to no more than d other vertices;
- Maximum spanning tree: weight greater than or equal to the weight of every other spanning tree;
- Dynamic MST: concerns the update after an edge weight change in the original graph or the insertion/deletion of a vertex;
- Minimum bottleneck spanning tree: not contain a spanning tree with a smaller bottleneck edge weight.

1. 以下叙述中，正确的是（ ）。
- A、只要无向连通图中没有权值相同的边，则其最小生成树唯一
 - B、只要无向图中有权值相同的边，则其最小生成树一定不唯一
 - C、从 n 个顶点的连通图中选取 $n-1$ 条权值最小的边，即可构成最小生成树
 - D、设连通图 G 含有 n 个顶点，则含有 n 个顶点、 $n-1$ 条边的子图一定是 G 的生成树

2. 【2012统考真题】下列关于最小生成树的叙述中，正确的是（ ）。

- I. 最小生成树的代价唯一
- II 所有权值最小的边一定会出现在所有的最小生成树中
- III 使用Prim算法从不同顶点开始得到的最小生成树一定相同
- IV 使用Prim算法和Kruskal算法得到的最小生成树总不相同

A、仅I B、仅II C、仅I、III D、仅II、IV

3. 【2015统考真题】求下面的带权图的最小生成树时，可能是Kruskal算法第2次选中但不是Prim算法（从 V_4 开始）第2次选中的边是（ ）。

- A、 (V_1, V_3) B、 (V_1, V_4)
- C、 (V_2, V_3) D、 (V_3, V_4)

